toXo

Task and Personal Finance Management Application

Technical Documentation

Complete guide to the architecture, features, and usage

Author: TheOverforge | github.com/TheOverforge

Technology Stack

Python 3.12 PyQt6 $\geq$ 6.7 SQLite v12

pyqtgraph $\geq$ 0.13 numpy pathlib datetime

March 13, 2026

Contents

1 Application Overview **2**
1.1 Key capabilities 2

2 Technologies Used **3**
2.1 Python 3.12 3
2.2 PyQt6 ≥ 6.7 3
2.2.1 Key PyQt6 classes 4
2.3 SQLite (via built-in `sqlite3`) 4
2.4 pyqtgraph ≥ 0.13 4
2.5 numpy 5

3 Project Architecture **5**
3.1 Feature Slice Design (FSD) 5
3.2 Directory structure 5
3.3 Centralized paths (`shared/config/paths.py`) 6
3.4 MainWindow architecture (mixin composition) 6
3.5 Right-panel pages (`QStackedWidget`) 7

4 Database Schema **7**
4.1 `tasks` table (18 fields) 7
4.2 `categories` table 8
4.3 `subtasks` table 8
4.4 Finance tables 8
4.4.1 `fin_accounts` 8
4.4.2 `fin_categories` 9
4.4.3 `fin_transactions` 9
4.4.4 `fin_budgets` 9
4.4.5 `fin_goals` 10
4.5 Schema migration history 10

5 Data Models **11**
5.1 Task 11
5.2 Category 11
5.3 Finance models 11

6 Service Layer **12**
6.1 TaskService 12
6.2 CategoryService 13
6.3 AnalyticsService 13
6.4 FinanceService 14
6.5 i18n — localization 14

7 User Interface **15**
7.1 Main window layout 15
7.2 Left panel 15
7.2.1 Search field 15
7.2.2 Filter buttons 16
7.2.3 Sorting 16
7.2.4 Category bar (`CategoryBar`) 16
7.2.5 Task list 16
7.2.6 Bottom panel 17
7.3 Task editor (page 1) 17
7.3.1 Description formatting toolbar 17
7.4 Analytics page (page 2) 18
7.5 Calendar page (page 4) 18
7.6 Finance page (page 5) 18
7.6.1 Overview 18
7.6.2 Transactions 19
7.6.3 Budgets 19
7.6.4 Goals 19
7.6.5 Accounts 19
7.6.6 Demo data (first run) 19
7.7 Command palette (Ctrl+K) 20

8 Functions and Features **20**
8.1 Task management 20
8.1.1 Creating a task 20
8.1.2 Inline rename 20
8.1.3 Mark as completed 20
8.1.4 Deletion 21
8.1.5 Delete undo 21
8.1.6 Duplication 21
8.2 Priorities 21
8.3 Reminders and deadlines 21
8.4 Tags 21
8.5 Subtasks 21
8.6 Categories 21
8.7 Archive 22
8.8 Bulk actions 22
8.9 Import 22
8.10 Language-dependent mock-data seeding 22
8.11 Audio notifications 22
8.12 Tutorial 22
8.12.1 Tutorial task 23

9 Hotkeys **23**

10 Custom Widgets **24**
10.1 TaskItemDelegate 24
10.2 `_CalGridWidget` 24

10.3 GlassFrame 25
10.4 CommandPalette 25
10.5 CategoryBar 25
10.6 `_FontPickerPanel` 25
10.7 CustomFontDialog 26
10.7.1 Loading fonts on startup 26
10.8 TextColorBar / InlineColorBar 26
10.9 FocusSaveLineEdit / FocusSaveTextEdit 27
10.10 HistoryMixin 27

11 Themes 27
11.1 Dark theme 27
11.2 Light theme 28
11.3 Glass theme (iOS blur-glass) 28
11.4 Finance color system 28
11.5 Windows DWM — title bar color 29
11.6 Button CSS classes 29

12 Settings 30

13 Testing 31

14 Known Behaviors and Limitations 31

1 Application Overview

toXo is a desktop application for personal task and finance management, developed in Python with PyQt6. The interface supports three visual themes: iOS Dark Glass (dark), Light, and Glass (semi-transparent). The main accent color is blue, `#0a84ff`.

1.1 Key capabilities

- **Full task CRUD** — create, view, edit, and delete tasks
- **Priorities** — four levels: none / low / medium / high
- **Pinning** — keep important tasks at the top of the list
- **Deadlines** — tray notifications one day before and on the due date
- **Reminders** — arbitrary reminder time with popup notification
- **Recurrence** — daily / weekly / monthly
- **Tags** — comma-separated labels; searchable with `#tag`
- **Subtasks** — checkbox list inside a task
- **Categories (folders)** — colored folders; drag-and-drop for tasks and folders
- **Filtering and search** — status, category, today's deadline, text, tag
- **Sorting** — by date, alphabetically, undone first, or manually
- **Archive** — completed tasks can be archived automatically after N days
- **Quick add** — top input field for instant task creation
- **Bulk actions** — select multiple items, then complete / move / delete
- **Analytics** — four chart types, KPI cards, streaks, and period comparison
- **Calendar page** — month grid with deadline markers and a daily bar chart
- **Finance** — transactions, budgets, goals, accounts; 5 tabs and 3 charts
- **Hotkeys** — full keyboard-driven workflow
- **Command palette** — Ctrl+K to find and execute any application command
- **Zoom** — Ctrl+wheel or Ctrl+/−/0
- **Delete undo** — Ctrl+Z restores the most recently removed task

- **Interactive tutorial** — first-run guided onboarding plus a pinned HTML tutorial task describing all features
- **Localization** — Russian and English, switchable in settings
- **System tray** — background operation with icon and context menu
- **Audio notifications** — 6 reminder sound presets
- **Three themes** — Dark, Light, Glass (iOS blur-glass)
- **Import** — CSV and JSON through the settings dialog
- **Custom fonts** — load personal `.ttf/.otf` fonts through the font picker panel

2 Technologies Used

2.1 Python 3.12

Primary implementation language. The project uses the following modern Python features:

- `from __future__ import annotations` — postponed evaluation of type annotations
- Union syntax `int | None` for optional types
- `dataclass` from `dataclasses` for data models
- `pathlib.Path` for path management
- `datetime` with `timezone.utc` for UTC-based timestamps
- `calendar` for month length calculations

2.2 PyQt6 $\geq$ 6.7

Qt 6 bindings for Python. The application relies on the following modules:

Module	Purpose
`PyQt6.QtWidgets`	UI widgets: main window, buttons, lists, dialogs, panels
`PyQt6.QtCore`	Qt core: signals, timers, settings, constants
`PyQt6.QtGui`	Graphics: QPainter, fonts, colors, icons, hotkeys
`QSystemTrayIcon`	System tray integration with menu and notifications
`QSettings`	Persistent user settings in registry / file storage

Module	Purpose
`QGraphicsDropShadowEffect`	Drop shadows under interface elements

2.2.1 Key PyQt6 classes

Class	Usage
`QMainWindow`	Root window of the application
`QSplitter`	Resizable divider between left and right panels; also used vertically in the editor
`QStackedWidget`	Switches between right-panel pages (6 pages)
`QListWidget`	Task list with a custom delegate
`QStyledItemDelegate`	Custom rendering of list rows
`QScrollArea`	Horizontal scrolling for categories; vertical scrolling for subtasks
`QDrag/QMimeData`	Drag-and-drop for tasks and categories
`QShortcut`	Keyboard shortcuts
`QTimer`	Deferred refreshes; reminder timer (every 60 s)
`QPainter`	Rendering the calendar grid and custom finance cards
`QToolButton`	"⋯" button for task context menus

2.3 SQLite (via built-in `sqlite3`)

The database is stored in:

- **Windows:** `%APPDATA%\toXo_app\tasks.sqlite`
- **Linux / macOS:** `~/.toXo_app/tasks.sqlite`

The schema supports incremental migrations through `PRAGMA user_version`. Current schema version: **v12**.

2.4 pyqtgraph $\geq$ 0.13

High-performance 2D charting library for Qt. It is used in three main areas:

- **Analytics page** — line chart, cumulative chart, weekday histogram, grouped bar chart
- **Calendar page** — created/completed bar chart by day of month
- **Finance page** — grouped income/expense bars and balance trend area chart

All charts use `pg.PlotWidget` with `hideButtons()`. After items are added, the code calls `getViewBox().enableAutoRange()`. It is important *not* to call `setXRange/setYRange` afterward, otherwise the plotted items can collapse into solid blocks.

2.5 numpy

Used for generating bar position arrays for `pg.BarGraphItem` and for chart calculations in the finance module.

3 Project Architecture

3.1 Feature Slice Design (FSD)

The project is organized according to the Feature Slice Design principle: each layer only knows about the layers below it.

`shared` → `entities` → `features` → `pages` → `widgets` → `app`

3.2 Directory structure

Project structure (FSD)

```
todo_app/
├── main.py                        # Thin wrapper: QApplication + MainWindow
├── requirements.txt
├── app/                           # Application level
│   ├── bootstrap.py               # MainWindow (assembly, mixin composition)
│   ├── styles/themes.py           # Dark / Light / Glass tokens + apply_app_style()
│   └── providers/ ...             # theme_provider, i18n_provider, settings_provider
├── pages/                         # Full-screen pages inside right_stack
│   ├── analytics/page.py          # AnalyticsPage (index 2)
│   ├── calendar/page.py           # CalendarPage (index 4)
│   └── finance/page.py            # FinancePage (index 5)
├── widgets/                       # Composite UI widgets
│   ├── command_palette/ui.py      # CommandPalette (Ctrl+K)
│   ├── finance_sidebar/ui.py      # Finance sidebar
│   └── tutorial/ui.py             # TutorialOverlay
├── features/                      # Business features (action-based slices)
│   ├── task/
│   │   ├── filter_tasks/ui.py     # FilterMixin
│   │   ├── edit_task/ui.py        # EditorMixin
│   │   ├── manage_tasks/ui.py     # TasksMixin
│   │   └── undo_redo/ui.py        # HistoryMixin
│   ├── category/manage_categories/ui.py  # CategoriesMixin
│   ├── finance/ ...               # Finance features
│   ├── settings/manage_settings/ui.py    # SettingsMixin
│   └── navigation/system/ui.py    # SystemMixin (tray, notifications, toast)
├── entities/                      # Domain models and repositories
│   ├── task/
│   │   ├── model.py               # dataclass Task
│   │   ├── repository.py          # TaskDbMixin (SQL)
│   │   ├── service.py             # TaskService
│   │   └── ui/task_list.py        # TaskItemDelegate, CategoryBar, TaskListWidget
│   ├── category/
│   │   ├── model.py / repository.py / service.py
│   ├── analytics/
```

```
│   │   ├── repository.py / service.py
│   └── finance/
│       ├── models.py             # FinAccount, FinCategory, Transaction, Budget, Goal
│       ├── repository.py         # FinanceDbMixin (+ _seed_finance_mock)
│       └── service.py            # FinanceService
├── shared/                       # Code with no dependencies on upper layers
│   ├── api/db/connection.py      # Database (combines all DbMixin classes)
│   ├── i18n/__init__.py          # tr(), set_language(), current_language()
│   ├── ui/widgets.py             # FocusSaveLineEdit, FocusSaveTextEdit, ...
│   ├── ui/dialogs.py             # ReminderDialog, DownwardCombo, CrispCheckBox
│   ├── config/paths.py           # PROJECT_ROOT, ASSETS_DIR, ICONS_DIR, ...
│   └── assets/
│       ├── icons/                # SVG icons
│       ├── images/               # PNG assets (app_icon, logo_text, ...)
│       └── sounds/               # WAV notification sounds
├── tests/
│   ├── unit/ ...
│   ├── integration/ ...
│   └── ui/ ...
└── docs/documentation.tex
```

3.3 Centralized paths (shared/config/paths.py)

```python
from pathlib import Path

PROJECT_ROOT: Path = Path(__file__).resolve().parent.parent.parent
ASSETS_DIR:  Path = PROJECT_ROOT / "shared" / "assets"
ICONS_DIR:   Path = ASSETS_DIR / "icons"
IMAGES_DIR:  Path = ASSETS_DIR / "images"
SOUNDS_DIR:  Path = ASSETS_DIR / "sounds"
```

All modules import path constants from this file instead of recomputing local paths such as `Path(__file__).parent...`.

3.4 MainWindow architecture (mixin composition)

`app/bootstrap.py` contains the `MainWindow` class, composed from seven mixins:

Mixin	File / Responsibility
`FilterMixin`	`features/task/filter_tasks/ui.py` — filtering, sorting, search
`EditorMixin`	`features/task/edit_task/ui.py` — task editor (right panel)
`TasksMixin`	`features/task/manage_tasks/ui.py` — create, delete, duplicate
`CategoriesMixin`	`features/category/manage_categories/ui.py` — categories
`SystemMixin`	`features/navigation/system/ui.py` — tray, notifications, zoom
`SettingsMixin`	`features/settings/manage_settings/ui.py` — settings dialog

Mixin	File / Responsibility
`HistoryMixin`	`features/task/undo_redo/ui.py` — undo stack, toast

3.5 Right-panel pages (`QStackedWidget`)

Index	Page	When shown
0	Empty	No task is selected
1	Editor	A task is selected in the list
2	Analytics	The "Analytics" button is pressed
3	Bulk actions	≥ 2 tasks are selected
4	Calendar	The "Calendar" button is pressed
5	Finance	The "Finance" item is chosen in the hamburger menu

4 Database Schema

4.1 tasks table (18 fields)

Field	Type	Description
`id`	INTEGER PK	Auto-increment identifier
`title`	TEXT NOT NULL	Task title
`description`	TEXT NOT NULL	Description / note (default: empty string)
`is_done`	INTEGER	0 = active, 1 = completed
`created_at`	TEXT	ISO UTC creation time
`completed_at`	TEXT	ISO UTC completion time (NULL if not completed)
`updated_at`	TEXT	ISO UTC last update time
`priority`	INTEGER	Priority: 0=none, 1=low, 2=medium, 3=high
`category_id`	INTEGER	FK to `categories.id` (NULL = uncategorized)
`remind_at`	TEXT	ISO UTC reminder time (NULL = no reminder)
`remind_shown`	INTEGER	0 = reminder not shown yet, 1 = already shown
`deadline_at`	TEXT	ISO UTC deadline (NULL = none)
`deadline_notified`	INTEGER	0 = no notification, 1 = "today" shown, 2 = "overdue" shown
`is_pinned`	INTEGER	1 = always pinned at the top of the list

Field	Type	Description
recurrence	TEXT	NULL / daily / weekly / monthly
tags	TEXT	Comma-separated tags, e.g. "work,urgent"
sort_order	INTEGER	Position for manual sorting (smaller = higher)
is_archived	INTEGER	1 = task is archived (hidden from the main view)

4.2 categories table

Field	Type	Description
id	INTEGER PK	Auto-increment identifier
name	TEXT UNIQUE	Category name
color	TEXT	HEX color string (#0a84ff)
sort_order	INTEGER	Display order
created_at	TEXT	ISO UTC creation time

4.3 subtasks table

Field	Type	Description
id	INTEGER PK	Auto-increment identifier
task_id	INTEGER NOT NULL	FK to tasks.id
title	TEXT NOT NULL	Subtask text
is_done	INTEGER	0 = active, 1 = completed
sort_order	INTEGER	Display order
created_at	TEXT	ISO UTC creation time

4.4 Finance tables

4.4.1 fin_accounts

Field	Type	Description
id	INTEGER PK	
name	TEXT	Account name
type	TEXT	cash / card / savings / invest
balance	REAL	Current balance
currency	TEXT	Currency symbol (₽, \$, €, ...)
color	TEXT	Card HEX color

Field	Type	Description
created_at	TEXT	ISO UTC timestamp

4.4.2 fin_categories

Field	Type	Description
id	INTEGER PK	
name	TEXT	Name (for example: “Food”, “Salary”)
type	TEXT	income / expense
color	TEXT	HEX color
icon	TEXT	Emoji icon
created_at	TEXT	ISO UTC timestamp

4.4.3 fin_transactions

Field	Type	Description
id	INTEGER PK	
type	TEXT	income / expense / transfer
amount	REAL	Amount
currency	TEXT	Currency code / symbol
category_id	INTEGER	FK to fin_categories.id
account_id	INTEGER	FK to fin_accounts.id
date	TEXT	Date in YYYY-MM-DD format
title	TEXT	Transaction description
note	TEXT	Optional note
tags	TEXT	Comma-separated tags
is_recurring	INTEGER	1 = recurring
recurring_rule	TEXT	Recurrence rule
created_at	TEXT	ISO UTC timestamp

4.4.4 fin_budgets

Field	Type	Description
id	INTEGER PK	
category_id	INTEGER	FK to fin_categories.id
limit_amount	REAL	Spending limit for the period
period	TEXT	monthly / weekly
created_at	TEXT	ISO UTC timestamp

4.4.5 fin_goals

Field	Type	Description
id	INTEGER PK	
name	TEXT	Goal name
target_amount	REAL	Target amount
current_amount	REAL	Amount saved so far
deadline	TEXT	Date in YYYY-MM-DD format
color	TEXT	HEX color
currency	TEXT	Currency
created_at	TEXT	ISO UTC timestamp

4.5 Schema migration history

Version	Changes
v0 → v1	+completed_at, updated_at, priority; created task_events table
v1 → v2	Created categories table; +category_id in tasks; added indexes
v2 → v3	+remind_at, remind_shown; added idx_tasks_remind
v3 → v4	+deadline_at, deadline_notified; added idx_tasks_deadline
v4 → v5	+is_pinned, recurrence; added idx_tasks_pinned
v5 → v6	Created subtasks table; added idx_subtasks_task
v6 → v7	+tags, sort_order; backfilled sort_order=id
v7 → v8	+is_archived; added idx_tasks_archived
v8 → v9	Created fin_accounts, fin_categories, fin_transactions, fin_budgets, fin_goals; seeded initial demo data (30 days of transactions)
v9 → v10	Cleared old finance demo data; reseeded with a richer set (90 days of transactions, more realistic values)
v10 → v11	Seeded 90 days of task history through _seed_task_history() for analytics demo purposes; added the tutorial task
v11 → v12	Added tasks with real deadlines for the Calendar page (overdue, today, future) through _seed_calendar_tasks()

5 Data Models

5.1 Task

Dataclass from `entities/task/model.py`:

```
@dataclass
class Task:
    id: int
    title: str
    description: str
    is_done: bool
    created_at: str              # ISO UTC
    completed_at: str | None = None
    updated_at: str | None = None
    priority: int = 0            # 0=none 1=low 2=med 3=high
    category_id: int | None = None
    remind_at: str | None = None
    remind_shown: bool = False
    deadline_at: str | None = None
    deadline_notified: int = 0   # 0/1/2
    is_pinned: bool = False
    recurrence: str | None = None  # daily/weekly/monthly
    tags: str = ""               # comma-separated
    sort_order: int = 0
    is_archived: bool = False
```

5.2 Category

Dataclass from `entities/category/model.py`:

```
@dataclass
class Category:
    id: int
    name: str
    color: str           # HEX, e.g. "#0a84ff"
    sort_order: int
    created_at: str
    task_count: int = 0  # calculated on query
```

5.3 Finance models

Dataclasses in `entities/finance/models.py`:

```
@dataclass
class FinAccount:
    id: int
    name: str
    type: str       # cash/card/savings/invest
    balance: float
    currency: str   # "RUB"/"USD"/"EUR" etc.
```

```python
    color: str
    created_at: str

@dataclass
class Transaction:
    id: int
    type: str       # income/expense/transfer
    amount: float
    currency: str
    category_id: int | None
    account_id: int | None
    date: str       # YYYY-MM-DD
    title: str
    note: str = ""
    tags: str = ""
    is_recurring: bool = False
    recurring_rule: str = ""
    created_at: str = ""

@dataclass
class Budget:
    id: int
    category_id: int
    limit_amount: float
    period: str     # monthly/weekly
    created_at: str

@dataclass
class Goal:
    id: int
    name: str
    target_amount: float
    current_amount: float
    deadline: str   # YYYY-MM-DD
    color: str
    currency: str
    created_at: str
```

6 Service Layer

6.1 TaskService

File: `entities/task/service.py`

Method	Description
`create_task(title,desc, cat_id)`	Creates a task and returns its `id`
`list_tasks()`	Returns all tasks, including archived ones

Method	Description
`get_task(id)`	Returns one task or `None`
`update_task(id, **fields)`	Updates an arbitrary set of fields
`set_done(id,is_done)`	Toggles completion; if `recurrence` is set, creates a new clone
`delete_task(id)`	Deletes a task
`restore_task(snap)`	Restores a task from a field snapshot (used for undo)
`duplicate_task(id)`	Duplicates a task and returns a new `id`
`archive_old_done(days)`	Moves completed tasks older than N days to the archive
`list_subtasks(task_id)`	Returns subtasks for a task
`add_subtask(task_id, title)`	Creates a subtask
`set_subtask_done(sub_id,v)`	Toggles a subtask checkbox
`delete_subtask(sub_id)`	Deletes a subtask

6.2 CategoryService

File: `entities/category/service.py`

Method	Description
`list_categories()`	Returns all categories ordered by `sort_order`, including `task_count`
`get_category(id)`	Returns a category by `id` or `None`
`create_category(name, color)`	Creates a category and returns its `id`
`update_category(id, name,color)`	Updates category name and color
`delete_category(id)`	Deletes a category; tasks remain with `category_id=NULL`
`reorder_categories(ids)`	Reorders categories based on a passed list of `id` values
`set_task_category(task_id,cat_id)`	Assigns a category to a task

6.3 AnalyticsService

File: `entities/analytics/service.py`

Method	Description
`kpi(days)`	Period KPIs: total_completed, pct, streak, avg_hours

Method	Description
`kpi_with_delta(days)`	KPI values plus delta vs previous period; also includes all-time max streak
`completed_per_day(days)`	Completed tasks by day `[(date, count)]`
`created_vs_completed(days)`	`[(date,created,completed)]`
`completed_by_weekday(days)`	By weekday 0(Mon)–6(Sun)
`cumulative_completed(days)`	Cumulative completion totals
`status_distribution(days)`	`{done:N,active:N}`
`tasks_for_day(date_str)`	`{created:[...],completed:[...]}`
`tasks_with_deadline_on(date_str)`	Tasks whose deadline falls on the specified date
`month_data(year,month)`	`{date:(created,completed)}`

6.4 FinanceService

File: `entities/finance/service.py`

Method	Description
`period_range(days)`	Returns `(date_from,date_to)`; `days=0` = current month; `days=-1` = current year
`transactions(days)`	Returns a list of `Transaction` objects for the selected period
`kpi(days)`	`{income,expenses,balance,net,avg_per_day}`
`expenses_by_category(days)`	`[(cat_name,color,amount)]`
`income_by_category(days)`	Same structure for income categories
`balance_trend(days)`	`[(date,balance)]` — cumulative balance trend
`budget_status()`	`[(Budget,cat,spent,pct)]`
`accounts()`	Returns all accounts
`goals()`	Returns all goals
`add_transaction(...)`	Creates a transaction
`add_account(...)`	Creates an account
`add_budget(...)`	Creates a budget
`add_goal(...)`	Creates a financial goal

6.5 i18n — localization

File: `shared/i18n/__init__.py`

```python
from shared.i18n import tr, set_language, current_language

tr("filter.all")            # -> "All"
tr("fin.income")            # -> "Income"
tr("bulk.selected",
   count=3, noun="tasks")  # named arguments
set_language("en")          # switch language
current_language()           # -> "ru" | "en"
```

The active language is stored in `QSettings` under the `"language"` key. It is loaded in `main.py` before `MainWindow` is created. Finance-related localization keys use the `fin.*` prefix (76 keys).

7 User Interface

7.1 Main window layout

Main window layout scheme

Top area

Search field + add button

Filters: *All / Active / Done / Today / Archive*

Task sorting

Categories (chips)

Quick add input

Central area

Task list

☐ Task 1
☐ Task 2
✓ Task 3
⋮

Right panel (`right_stack`)

— empty / editor
— analytics / calendar
— finance / bulk actions

Bottom area

Actions: *Delete, Analytics, Calendar, ▯ Menu, Settings*

Counters: *Shown N, Completed N*

7.2 Left panel

7.2.1 Search field

SearchBox. Filters tasks in real time by `title` and `description`. The **#tag** prefix filters by the `tags` field. Hotkey: **Ctrl+F**.

7.2.2 Filter buttons

Button	Behavior
All	All non-archived tasks (active + completed)
Active	`is_done = 0` and `is_archived = 0`
Completed	`is_done = 1` and `is_archived = 0`
Today	Tasks due today or already overdue
Archive	`is_archived = 1`

7.2.3 Sorting

Mode	Description
Newest	Descending by `created_at`
Oldest	Ascending by `created_at`
A → Z	Alphabetical by title
Undone first	Active tasks first (newer first), then completed tasks
Manual	Drag-and-drop inside the list; persisted in `sort_order`

Pinned tasks (`is_pinned = 1`) are always rendered above the rest.

7.2.4 Category bar (`CategoryBar`)

A horizontally scrollable strip. Contains the "All Tasks" chip, colored category chips, and a "+" button. Supports drag-and-drop for both tasks and category chips.

7.2.5 Task list

`TaskListWidget` with the `TaskItemDelegate`. Each row includes:

- Colored category dot (6×6 px)
- Round checkbox (22×22 px)
- Priority marker (blue / yellow / red)
- Pin icon (star)
- Title (strikethrough when completed)
- Deadline `DD.MM` or the creation date
- Subtask counter (`X/N`) if the task has subtasks

7.2.6 Bottom panel

Element	Description
“Delete”	Deletes selected tasks
“Analytics”	Switches to the analytics page (index 2)
“Calendar”	Switches to the calendar page (index 4)
Hamburger menu	Opens a menu containing Finance and other entries
“Settings”	Opens a floating settings dialog
Counter	“Shown: N | Completed: N”

7.3 Task editor (page 1)

- **EditorTitle** — 28 px bold field; saves on focus loss or Enter
- **“…”** — task context menu (duplicate, export, command palette)
- **Priority pill buttons** — none / low / medium / high
- **Pin** control and recurrence dropdown
- **Remind / Deadline** — opens `ReminderDialog`
- **Tags** — comma-separated input field
- **EditorDesc** — multiline text editor with auto-save on focus loss
- **Subtasks** — section below (Enter adds one, removes one)

7.3.1 Description formatting toolbar

The formatting toolbar sits above the `EditorDesc` field:

- **Font button** (`DescFontCombo`) — shows the current font; click toggles `_FontPickerPanel`
- **Size buttons** (`DescSizeBtn`) — decrease / increase font size by 1 pt
- **“A ▾”** inside `TextColorBar` — another button that opens `_FontPickerPanel`
- **InlineColorBar** — text and highlight color panel, centered exactly within the toolbar row by symmetric spacer widgets

All popup panels (`_FontPickerPanel`, `TextColorBar`, `InlineColorBar`) are styled with a dark translucent background (`rgba(10,14,24,0.82)`) and a soft frame in Dark/Glass theme, while Light theme uses a bright variant.

7.4 Analytics page (page 2)

Period switcher: **7 / 14 / 30 days** plus a custom range.

KPI cards: Completed (with delta), Completion %, Current streak, Max streak, Average time.

Charts:

- Completed by day — line chart
- Created vs. completed — grouped bar chart
- Cumulative completed — area chart
- By weekday — histogram Mon–Sun

7.5 Calendar page (page 4)

Horizontal splitter (380 px / 620 px):

Left side — month grid rendered with `QPainter`: ‹/› buttons, day cells, green dot = deadline exists, blue fill = selected day.

Right side — list of tasks with deadlines on the selected day + "+ Add task" button + bar chart for the current month.

Drag-and-drop from the task list onto a calendar cell sets the deadline to 09:00 UTC.

7.6 Finance page (page 5)

How to open it

Click ☰ (hamburger menu) → "Finance". Clicking the entry again closes the page.

The page contains 5 tabs:

7.6.1 Overview

- KPI cards: Balance (blue), Income (green), Expenses (red), Net income, Avg/day
- "Income vs Expenses" chart — grouped bars by day/week (`pg.BarGraphItem`)
- Two QPainter donut charts — expenses by category / income by category
- "Balance trend" chart — area chart (`pg.PlotDataItem` with `fillLevel=0`)
- Latest 8 transactions

Period switcher: **7d / 30d / 90d / Month / Year**. Secondary text color is returned by `_ts()`. The function outputs a hex string dependent on the current theme: Light theme uses `_THEMES["light"]["text_sec"]`, Glass uses `#bdd0ec`, and Dark uses `#c0c0c8`. Hex output (rather than CSS `rgba`) is required for compatibility with pyqtgraph (`pg.setConfigOptions`) and `QColor(str)`.

7.6.2 Transactions

Filter by type (All / Income / Expenses / Transfers) plus search. Scrollable list of transaction rows.

7.6.3 Budgets

Two-column grid of budget cards. Each card shows the category, limit, spent value, progress bar (red when overspent), and a status badge. Includes a "+ Add budget" button.

7.6.4 Goals

Grid of goal cards with circular progress (QPainter arc). Each card shows the goal name, current amount / target, deadline, and percentage.

7.6.5 Accounts

Two-column grid of account cards with a colored accent stripe. The page header shows the total balance. Includes a "+ Add account" button.

7.6.6 Demo data (first run)

On the first run — or when the "Load test data" button is pressed in settings — the application calls `_seed_finance_mock()`. Data is generated in the current application language (`current_language()`).

Russian (RU):

- 3 accounts: localized Cash / Main Card / Savings equivalents with RUB balances (12,000 / 98,560 / 14,000)
- 10 categories: 7 expense + 3 income categories, all localized in Russian
- 90 days of transactions (salary, subscriptions, food, transport, freelance, ...)
- 3 budgets: Food RUB 15,000, Transport RUB 5,000, Entertainment RUB 8,000
- 4 goals: New Monitor, Vacation, Car, Emergency Fund — localized in Russian

English (EN):

- 3 accounts: Cash ($130), Main Card ($1,090), Savings ($155)
- 10 categories with English names (Food, Transport, Subscriptions, ...)
- 90 days of USD transactions (original RUB amounts divided by 90)
- 3 budgets: Food / Transport / Entertainment
- 4 goals: "New MacBook", "Trip to Asia", "Car", "Emergency fund"

7.7 Command palette (Ctrl+K)

`widgets/command_palette/ui.py`. A frameless dialog with a search field and a command list (roughly 26 commands):

- Tasks: create, delete, duplicate, mark complete
- Filters and sorting modes
- Open analytics / calendar / finance / settings
- Change language, theme, zoom
- Export / import, bulk actions

Favorite commands (star-marked) are stored in `QSettings "command_palette/favorit` as a JSON list and displayed first. Right-clicking inside `FocusSaveTextEdit` (task description) also opens the palette.

8 Functions and Features

8.1 Task management

8.1.1 Creating a task

Ways to create a task

- **Ctrl+N** or the **"+"** button near the search field
- **QuickAdd field** — type the title and press Enter
- **Calendar page** — "+ Add task" (creates a task with a deadline on the selected day)

8.1.2 Inline rename

F2 or a **double-click** on a task. Enter or clicking outside saves; Esc cancels.

8.1.3 Mark as completed

Click the checkbox, press **Ctrl+D**, or use the context menu entry "Complete". If `recurrence` is set, a new task clone is created automatically with a shifted deadline.

8.1.4 Deletion

Use the **Delete** key, the “Delete” button, or the context menu action. By default, deletion is confirmed through a dialog.

8.1.5 Delete undo

Ctrl+Z restores the task through `svc.restore_task(snap)`. Snapshots are stored in the `HistoryMixin` stack (up to 50 entries).

8.1.6 Duplication

Ctrl+Shift+D or context menu → “Duplicate”.

8.2 Priorities

Four levels, represented by a colored marker in the task row:

- **None** — no marker
- **Low** — blue
- **Medium** — yellow
- **High** — red

8.3 Reminders and deadlines

A timer checks reminders every 60 seconds. Notifications are delivered through the system tray with sound. Deadlines have two levels of notification: one day before, and on the day itself / overdue. Tasks due today are included in the “Today” filter.

8.4 Tags

Comma-separated field. Searching **#tag** in the SearchBox filters tasks by tag.

8.5 Subtasks

Enter adds one, the checkbox marks it complete, and removes it. The main list shows an `X/N` counter.

8.6 Categories

Create with the “+” button; 8 colors are available. Assignment can be done by drag-and-drop or the context menu entry “Move to...”. Reordering is done by dragging one chip onto another.

8.7 Archive

Completed tasks older than N days can be archived automatically on startup. Both the setting and the value of N are configurable in the settings dialog.

8.8 Bulk actions

When ≥ 2 tasks are selected (Ctrl+A or Ctrl+click), the right panel switches to the "Bulk actions" page (index 3): complete / uncomplete, move to category, delete.

8.9 Import

The settings dialog contains "Import CSV" and "Import JSON" actions. CSV must contain at least the `title` column; JSON must be an array of objects.

8.10 Language-dependent mock-data seeding

When the "Load test data" button is pressed (Settings → Danger zone), two methods are called:

- `db._seed_task_history()` — creates roughly 90 days of task history grouped by categories. It reads `current_language()` and generates titles in RU or EN. At the end it also calls `_seed_tutorial_task()`.
- `db._seed_finance_mock()` — creates accounts, categories, 90 days of transactions, budgets, and goals. It is also language-aware: EN uses dollar values and English labels.

Both methods are also triggered from migrations (v11 and v9→v10 respectively), so existing databases receive demo content during upgrades as well.

8.11 Audio notifications

Six presets are stored in `shared/assets/sounds/`:

- `toXo_default.wav`, Bright Minimal, Deep Luxe, Double Check, Ultra Soft, Warm Tap Chime

The selected file is stored in settings, while the path is built through `SOUNDS_DIR` from `shared/config/paths.py`.

8.12 Tutorial

Runs on first startup (`QSettings "tutorial_shown" = False`). `TutorialOverlay` (`widgets/tutorial/ui.py`) is a translucent overlay that highlights widgets and provides "Back" / "Next" / "Skip" controls. Total: 9 steps.

8.12.1 Tutorial task

On first startup, if there are no tasks yet, the application automatically creates a pinned tutorial task (`_seed_tutorial_task()` in `shared/api/db/connection.py`

- Detects the current language through `current_language()` and creates content in RU or EN
- Rich HTML description with colored sections (hotkeys, filters, finance, analytics, etc.) in a `QTextEdit`-compatible format with inline CSS
- Task is pinned (`is_pinned=1`), tagged `"tutorial"`, priority 0
- First-run flag: `QSettings "tutorial_task_seeded"` — set to `True` once the task is created

The tutorial task is also included when "Load test data" is pressed in settings (called at the end of `_seed_task_history()`).

9 Hotkeys

Shortcut	Action
Ctrl+N	Create a new task
Ctrl+F	Move focus to the search field
Ctrl+S	Force-save the current task
Ctrl+D	Toggle completion for the selected task
Ctrl+Shift+D	Duplicate the selected task
Ctrl+Z	Undo the latest deletion (restore task)
Ctrl+Y	Redo the undone action
Ctrl+K	Open the command palette
Ctrl+Up	Move to the previous task
Ctrl+Down	Move to the next task
Ctrl+A	Select all tasks
F2	Rename task inline
F11	Toggle fullscreen mode
Delete	Delete the selected task (if focus is not inside a text editor)
Escape	Back in calendar / close page / clear selection
Ctrl + "+" / "="	Zoom in
Ctrl + "-"	Zoom out
Ctrl + 0	Reset zoom to 100%
Ctrl + mouse wheel	Zoom in / out

Escape behavior

1. If the "←" button (back to calendar) is **visible** — open the Calendar page
2. Otherwise, if **Analytics** is open (page 2) — close it
3. Otherwise, if **Calendar** is open (page 4) — close it
4. Otherwise, if **Finance** is open (page 5) — close it
5. Otherwise — save the task, clear selection, and switch to the empty page

10 Custom Widgets

10.1 TaskItemDelegate

`entities/task/ui/task_list.py`. Custom delegate for `QListWidget`:

- `paint()` — category dot, checkbox, priority, star icon, title, counter, date
- `editorEvent()` — intercepts clicks in the checkbox area
- `createEditor()` — `QLineEdit` for inline editing
- `sizeHint()` — row height 44 px

Important: PyQt6 CheckState

`index.data(Qt.ItemDataRole.CheckStateRole)` returns an `int` (0, 1, 2), not a `Qt.CheckState`. The correct comparison is `val == 2`, not `val == Qt.CheckState.Checked`. For `setData`, `Qt.CheckState.Checked` is still used.

Important: QMouseEvent in PyQt6

Use `event.position()` (returns `QPointF`), not the deprecated `event.pos()`.

10.2 _CalGridWidget

`pages/calendar/page.py`. Rendered with `QPainter`:

- Cell constants: `_CELL_W=58, _CELL_H=54, _HDR_H=44, _DOW_H=22`
- `day_selected(object)` signal — emits `datetime.date`
- `tasks_dropped_on_day(list, object)` — drag-and-drop tasks onto a day
- `set_deadline_days(set[str])` — refreshes the green deadline markers

10.3 GlassFrame

Base class for finance cards (`pages/finance/page.py`). In Glass theme it draws a blurred background through `get_glass_blur_region()`, then applies tint and frame manually via `QPainter`. In Dark and Light themes it delegates painting to `QFrame.paintEvent()` so QSS handles the background.

Subclasses: `FinKpiCard`, `_DonutWidget`, `_TxRow`, `_AccountCard`, `_BudgetCard`, and `_GoalCard`.

10.4 CommandPalette

`widgets/command_palette/ui.py`. A `QDialog` with `FramelessWindowHint` and `WA_TranslucentBackground`. It opens centered over the `centralWidget` and supports star-marked favorite commands plus section separators.

10.5 CategoryBar

Horizontal category panel. Its signals are listed below.

Signal	Payload
`category_selected`	`int id` (-1 = "All")
`category_rename`	`int id`
`category_delete`	`int id`
`add_category`	none
`category_reordered`	`(dragged_id,target_id)`
`task_dropped`	`([task_ids],category_id)`

10.6 _FontPickerPanel

`shared/ui/widgets.py`. Floating frameless panel (`Tool | FramelessWindowHint | WA_TranslucentBackground`) used to choose the font for a task description.

- The font list is built from `get_font_presets()` and includes built-in fonts plus custom fonts stored in the `QSettings` key `"custom_fonts"`.
- The bottom of the list contains the `FontPickerAdd` button: "+ Add custom font...". It opens `CustomFontDialog`.
- **Closing behavior:** clicking outside the panel is handled by a global `eventFilter`; clicks on the trigger button are forwarded back to the button handler using the `source_btn` pattern.
- `show_animated(x,y,source_btn)` accepts a reference to the trigger button so the panel does not close when the same button is pressed again.
- `apply_theme(theme)` repaints the panel and its buttons for the active theme.

- `refresh_fonts()` rebuilds the font rows after a font is added or removed.

Important: `source_btn` pattern

The global event filter closes the panel on the first click outside it. If that click lands on the trigger button, the filter must allow it to pass (`returnFalse`) so the button handler can toggle visibility itself. Otherwise the panel closes and immediately reopens.

10.7 CustomFontDialog

`shared/ui/widgets.py`. Dialog for managing custom fonts.

- **Add button:** opens a `QFileDialog` for `*.ttf` and `*.otf`. The selected file is copied into `shared/assets/fonts/`. Metadata is then stored in `QSettings("todo_app","todo_mvp")` under the keys `"custom_font_files"` (filenames) and `"custom_fonts"` (font families).
- After a successful load, `QFontDatabase.addApplicationFont()` registers the font for the current session. The family name is read from `applicationFontFamilies()`.
- On error (when `addApplicationFont()` returns -1), the copied file is removed and `QMessageBox.warning()` is shown.
- The **X** button next to each font removes it from both `QSettings` and the local `fonts/` directory.
- The `fonts_changed` signal notifies `_FontPickerPanel.refresh_fonts()`.

10.7.1 Loading fonts on startup

`load_custom_fonts()` is called in `main.py` before `set_language()`:

```
def load_custom_fonts() -> None:
    _CUSTOM_FONTS_DIR.mkdir(parents=True, exist_ok=True)
    for fname in QSettings("todo_app", "todo_mvp").value(
            "custom_font_files", [], type=list):
        p = _CUSTOM_FONTS_DIR / fname
        if p.exists():
            QFontDatabase.addApplicationFont(str(p))
```

10.8 TextColorBar / InlineColorBar

`shared/ui/widgets.py`. Floating color-formatting panels for the `EditorDesc` field.

TextColorBar opens when text is selected in `EditorDesc`. It contains text-color buttons, background/highlight buttons, and the “A ▾” trigger that opens `_FontPickerPanel`.

InlineColorBar is the persistent toolbar panel above `EditorDesc`. It is centered by using symmetrical spacer widgets:

```
_left_w  = QWidget()  # stretch=1, contains buttons + addStretch()
inline_color_bar = InlineColorBar(...)  # stretch=0
_right_w = QWidget()  # stretch=1, empty
bar_row.addWidget(_left_w, 1)
bar_row.addWidget(inline_color_bar, 0, AlignVCenter)
bar_row.addWidget(_right_w, 1)
```

Both panels share the same background style: `rgba(10,14,24,0.82)` in Dark/Glass theme and a light equivalent in Light theme.

10.9 FocusSaveLineEdit / FocusSaveTextEdit

`shared/ui/widgets.py`. Extensions of standard widgets with `on_focus_out` and `on_enter`. `FocusSaveTextEdit` also emits `context_menu_requested(QPoint)`: it intercepts the standard context menu and opens the command palette instead.

10.10 HistoryMixin

`features/task/undo_redo/ui.py`. Undo stack with up to 50 entries. A task snapshot is stored as `t.__dict__.copy()`. Restoration is performed through `svc.restore_task(snap)`. Toast notifications are shown through `_show_toast(text)` and stay at the bottom of the window for 2.2 seconds.

Drag-and-drop MIME types:

- One task: `application/x-task-id` (string)
- Multiple tasks: `application/x-task-ids` (comma-separated)

11 Themes

Theme file: `app/styles/themes.py`. Main module functions:

- `set_theme(name)`
- `current_theme()`
- `apply_app_style(app)`
- `apply_button_shadows(window)`

11.1 Dark theme

Variable	Value	Purpose
`bg_dark`	`#0d0d0f`	Main window background
`bg_panel`	`rgba(28,28,30,0.92)`	Glass-like panel background

Variable	Value	Purpose
glass	rgba(255,255,255, 0.07)	Button background
accent	#0a84ff	iOS-style violet accent
danger	#ff453a	Red accent
text	#f5f5f7	Primary text
text_sec	#98989d	Secondary text

11.2 Light theme

Variable	Value	Purpose
bg_dark	#E8EFF8	Soft lavender background
bg_panel	#F7F8FB	Light panels
accent	#3B82F6	Violet accent
text	#1A1F2E	Dark text
text_sec	#52607A	Secondary text
separator	#D8DFE9	Separators

11.3 Glass theme (iOS blur-glass)

Variable	Value	Purpose
bg_dark	#232c3a	Deep violet background
bg_panel	#1d2430	Panel base color
glass	rgba(255,255,255, 0.12)	Frosted-glass surface
accent	#5ea2ff	Brighter violet accent

Blur effect: `install_glass_bg(w)` installs a hidden widget named `_GlassBg`. It renders through `QGraphicsBlurEffect(radius=20)` and caches the result in `_blur_cache`. Cards based on `GlassFrame` call `get_glass_blur_region()` to obtain a blurred fragment for painting.

11.4 Finance color system

The helper function `_fc(key)` returns a theme-dependent color:

Key	Dark/Glass	Light
"inc"	#30d158 (green)	#52d38a
"exp"	#ff453a (red)	#ff6b6b
"bal"	#0a84ff (blue)	#5ea2ff
"goal"	#bf5af2 (purple)	#c27cff
"xfer"	#ff9f0a (orange)	#ffb84d

11.5 Windows DWM — title bar color

A dark title bar is applied in Dark and Glass theme:

```python
# main.py
def _set_dark_titlebar(window, dark: bool = True):
    import ctypes
    dwm = ctypes.windll.dwmapi
    DWMWA_USE_IMMERSIVE_DARK_MODE = 20
    mode = ctypes.c_int(1 if dark else 0)
    dwm.DwmSetWindowAttribute(hwnd, 20, byref(mode), sizeof(mode))
    DWMWA_CAPTION_COLOR = 35
    color = ctypes.c_uint32(0x000F0D0D if dark else 0x00FBF6F3)
    dwm.DwmSetWindowAttribute(hwnd, 35, byref(color), sizeof(color))
```

11.6 Button CSS classes

ObjectName	Description
`FilterBtn`	Pill filter button — glass style; `:checked` = blue fill
`NewBtn`	Round “+” button (36×36 px) with glow shadow
`DangerBtn`	Red pill button (delete)
`TodayBtn`	Red “Today” pill button
`ArchiveBtn`	Yellow/orange archive pill button
`AnalyticsBtn`	Blue analytics pill button
`CalendarBtn`	Green calendar pill button
`AccentBtn`	Blue “add” pill button with white text
`SettingsBtn`	Gray settings pill button
`TaskMenuBtn`	Small glass “⋯” editor button
`LangBtn`	Language / theme selector button in settings
`DescFontCombo`	Current description font button; click → `_FontPickerPanel`
`DescSizeBtn`	±1 pt buttons for description font size
`FontPickerAdd`	“+ Add custom font...” button at the bottom of `_FontPickerPanel`
`NeutralBtn`	Neutral pill button (blue outline, no fill) — used in the settings dialog danger zone
`FinKpiCard`	Finance KPI card
`FinTransactionRow`	Transaction row
`FinAccountCard`	Account card
`FinBudgetCard`	Budget card
`FinGoalCard`	Goal card
`FinBudgetBar`	Budget progress bar (normal)

ObjectName	Description
`FinBudgetBarDanger`	Budget progress bar (overspent, red)

12 Settings

The settings dialog is a floating `QDialog` (not part of `right_stack`). Its size is 500×720 with vertical scrolling through `QScrollArea`.

Section	Content
Language	“RU” / “EN” buttons; instant switching through `retranslate()`
Theme	“Dark” / “Light” / “Glass” buttons; applied immediately
Zoom	Slider 70–150%; label with current value
Delete confirmation	Toggle that enables/disables the confirmation dialog
Auto-archive	Toggle + input for the number of days
Notification sound	Preset dropdown + “Play” button
Import	“Import CSV” / “Import JSON” buttons
Danger zone	“Load test data” — seeds task and finance demo data in the current language; “Reset formatting” — removes HTML formatting from all task descriptions; “Delete all data” — fully clears the database and then recreates the tutorial task

All settings are stored in `QSettings("todo_app","todo_mvp")`. Important keys include:

- `"language"`, `"theme"`, and `"zoom"`
- `"confirm_delete"`, `"auto_archive_enabled"`, and `"auto_archive_days"`
- `"notification_sound"`
- `"custom_fonts"` (font families)
- `"custom_font_files"` (filenames stored in `shared/assets/fonts/`)
- `"tutorial_task_seeded"` (first-run tutorial flag)

Important: `QSettings` without arguments

Calling `QSettings()` without arguments uses a *different* registry/storage path than `QSettings("todo_app","todo_mvp")`. Every settings access in the codebase must therefore use the named constructor; otherwise values will not be shared between modules.

13 Testing

Test structure inside `tests/`:

- `tests/unit/` — unit tests for services and models
- `tests/integration/` — integration tests against real SQLite (in-memory)
- `tests/ui/` — widget and delegate tests

14 Known Behaviors and Limitations

Important technical notes

- **pyqtgraph auto-range:** after `enableAutoRange()` you must not call `setXRange()` or `setYRange()`. Those disable auto-scaling and charts may render as solid blocks.
- **Lazy chart initialization:** `_ensure_charts()` is called on the first `refresh()` to avoid pyqtgraph crashes inside a hidden `QStackedWidget`.
- **Circular imports:** `entities/__init__.py` exports *only* models such as `Task` and `Category`. Services are imported directly, for example `fromentities.task.serviceimportTaskService`, otherwise a cycle appears through `shared.api.db.connection`.
- **Resource paths:** always import from `shared.config.paths`; do not recompute `Path(__file__).parent...` in each module.
- **Windows DWM:** `_set_dark_titlebar` only works after `w.showMaximized()`; call it afterward.
- **QSettings without arguments:** `QSettings()` and `QSettings("todo_app","todo_mvp")` write to different registry sections. Every module must use the named constructor.
- **Opacity on QLabel:** CSS property `opacity:0.5` does not work on `QLabel` in Qt. Use `color:rgba(r,g,b,a)` for semi-transparent text.
- **Hamburger debounce:** `QMenu.exec()` blocks the event loop, so after the menu closes the button click can be processed again. Anti-

bounce is implemented through `_hamburger_closed_at` plus a 250 ms guard at the start of `_show_hamburger_menu`.

- **`_FontPickerPanel` transparency:** to get correct window alpha for the tool-style popup, `WA_TranslucentBackground` must be enabled. Rounded corners are created through `setMask(QRegion)` inside `resizeEvent`.
- **Finance `_ts()` output:** helper `_ts()` in `pages/finance/page.py` returns a hex string such as `#c0c0c8`, not CSS `rgba()`. `QColor(str)` and `pyqtgraph` accept hex or named colors only.